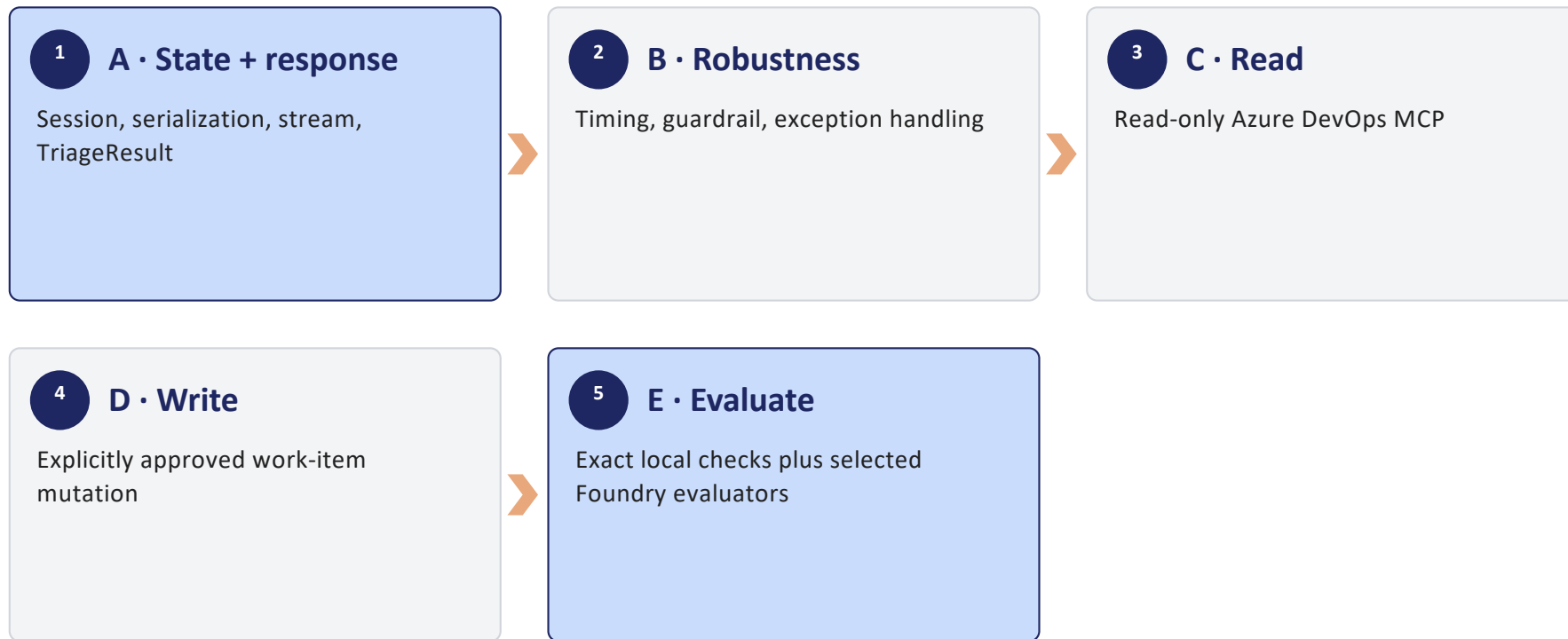


Day 3 Lab Kickoff

Preview the lab you're about to build

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/conversations/session?tabs=python> · +2 in notes

Lab architecture



Part A + B: runtime foundation

Part A

- Create and reuse AgentSession
- Serialize and restore with to_dict/from_dict
- Stream display updates
- Finalize a typed TriageResult

Part B

- Add logging/timing middleware
- Short-circuit one blocked request
- Handle a classified exception
- Demonstrate a bounded retry policy

Part C + D: read before write

Part C · Read-only

- X-MCP-Toolsets: wit
- X-MCP-Readonly: true
- wit_work_item with get or my
- Verify result against the workshop project

Part D · Approved write

- Allow wit_work_item_write
- Use create or update
- Show exact arguments before approval
- Read again to verify the mutation

Part E: evaluate the tool contract

1 Golden cases

Read, approved write, rejected write, and no-tool request



2 Expected calls

Tool name plus action and key arguments



3 Local checks

Presence and argument subset match

4 Repetitions

Observe consistency across independent runs



5 FoundryEvals

Add tool selection/input accuracy where available

Lab prerequisites

Day 2 baseline

Working docs assistant and
FoundryChatClient

Azure DevOps

Entra-backed Services org plus
dedicated workshop project

Permissions

Read access first; separately approved
write access

Foundry evals

Project client and judge model
deployment

Fixtures

Known work item IDs and a
reset/cleanup plan

Definition of done and guardrails

Area	Done when	Guardrail
Session	Restored turn retains intended state	Ownership mapping verified
Structured stream	UI updates, final typed value	No partial JSON actions
Middleware	Guard and failure path are observable	Retry is bounded
ADO MCP	Read succeeds; write requires approval	Dedicated project only
Evaluation	Expected tool/action/args are reported	No universal pass threshold claimed

Takeaways



You build read-only behavior before approved writes.



You'll see this five-part structure again in labs/day3/, then work through it yourself.



You compose Day 3's primitives across five stages: state and response, robustness, read, write, and evaluation.



You confirm prerequisites — a dedicated Azure DevOps project, a Foundry evaluation project, and known work-item fixtures — before you start the lab.